



 Latest updates: <https://dl.acm.org/doi/10.1145/3729357>

RESEARCH-ARTICLE

Zero-Shot Cross-Domain Code Search without Fine-Tuning

KEYU LIANG, Zhejiang University, Hangzhou, Zhejiang, China

ZHONGXIN LIU, Zhejiang University, Hangzhou, Zhejiang, China

CHAO LIU, Chongqing University, Chongqing, China

ZHIYUAN WAN, Zhejiang University, Hangzhou, Zhejiang, China

DAVID LO, Singapore Management University, Singapore City, Singapore

XIAOHU YANG, Zhejiang University, Hangzhou, Zhejiang, China

Open Access Support provided by:

Chongqing University

Zhejiang University

Singapore Management University

Published: 19 June 2025
Accepted: 01 April 2025
Received: 25 February 2025

[Citation in BibTeX format](#)

Zero-Shot Cross-Domain Code Search without Fine-Tuning

KEYU LIANG, Zhejiang University, China

ZHONGXIN LIU*, Zhejiang University, China

CHAO LIU, Chongqing University, China

ZHIYUAN WAN, Zhejiang University, China

DAVID LO, Singapore Management University, Singapore

XIAOHU YANG, Zhejiang University, China

Code search is a crucial task in software engineering, aiming to retrieve code snippets that are semantically relevant to a natural language query. Recently, Pre-trained Language Models (PLMs) have shown remarkable success and are widely adopted for code search tasks. However, PLM-based methods often struggle in cross-domain scenarios. When applied to a new domain, they typically require extensive fine-tuning with substantial data. Even worse, the data scarcity problem in new domains often forces these methods to operate in a zero-shot setting, resulting in a significant decline in performance. RAPID, which generates synthetic data for model fine-tuning, is currently the only effective method for zero-shot cross-domain code search. Despite its effectiveness, RAPID demands substantial computational resources for fine-tuning and needs to maintain specialized models for each domain, underscoring the need for a zero-shot, fine-tuning-free approach for cross-domain code search.

The key to tackling zero-shot cross-domain code search lies in bridging the gaps among domains. In this work, we propose to break the query-code matching process of code search into two simpler tasks: query-comment matching and code-code matching. We first conduct an empirical study to investigate the effectiveness of these two matching schemas in zero-shot cross-domain code search. Our findings highlight the strong complementarity among the three matching schemas, i.e., query-code, query-comment, and code-code matching. Based on the findings, we propose CodeBridge, a zero-shot, fine-tuning-free approach for cross-domain code search. Specifically, CodeBridge first employs zero-shot prompting to guide Large Language Models (LLMs) to generate a comment for each code snippet in the codebase and produce a code for each query. Subsequently, it encodes queries, code snippets, comments, and the generated code using PLMs and assesses similarities through three matching schemas: query-code, query-comment, and generated code-code. Lastly, CodeBridge leverages a sampling-based fusion approach that combines these three similarity scores to rank the final search outcomes. Experimental results show that our approach outperforms the state-of-the-art PLM-based code search approaches, i.e., CoCoSoDa and UniXcoder, by an average of 21.4% and 24.9% in MRR, respectively, across three datasets. Our approach also yields results that are better than or comparable to those of the zero-shot cross-domain code search approach RAPID, which requires costly fine-tuning.

*Zhongxin Liu is the corresponding author and is also with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security

Authors' Contact Information: [Keyu Liang](mailto:liangkeyu@zju.edu.cn), The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, liangkeyu@zju.edu.cn; [Zhongxin Liu](mailto:liu_zx@zju.edu.cn), The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, liu_zx@zju.edu.cn; [Chao Liu](mailto:liu_chao@cqu.edu.cn), School of Big Data and Software Engineering, Chongqing University, Chongqing, China, liu_chao@cqu.edu.cn; [Zhiyuan Wan](mailto:wanzhiyuan@zju.edu.cn), The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, wanzhiyuan@zju.edu.cn; [David Lo](mailto:davidlo@smu.edu.sg), School of Computing and Information Systems, Singapore Management University, Singapore, Singapore, davidlo@smu.edu.sg; [Xiaohu Yang](mailto:yangxh@zju.edu.cn), The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China, yangxh@zju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2025/7-ARTFSE087

<https://doi.org/10.1145/3729357>

CCS Concepts: • **Software and its engineering** → **Software development techniques**.

Additional Key Words and Phrases: Code Search, Pretrained Language Models, Zero-Shot Learning, Cross-Domain

ACM Reference Format:

Keyu Liang, Zhongxin Liu, Chao Liu, Zhiyuan Wan, David Lo, and Xiaohu Yang. 2025. Zero-Shot Cross-Domain Code Search without Fine-Tuning. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE087 (July 2025), 23 pages. <https://doi.org/10.1145/3729357>

1 Introduction

Code search aims to retrieve the code snippets that are semantically relevant to a provided natural language query from a codebase. It is one of the most frequent activities in software development [Xia et al. 2017], and can greatly enhance the efficiency of developers by assisting them in reusing the code from existing code repositories [Liu et al. 2021a; Sachdev et al. 2018]. In the era of large language models (LLMs), code search has gained new significance as a key component in retrieval-augmented generation (RAG) [Lewis et al. 2020] and in-context learning [Brown et al. 2020] which are widely used to improve tasks like code generation [Patel et al. 2024; Zhang et al. 2023]. To this end, many approaches have been proposed to improve the effectiveness and efficiency of code search [Feng et al. 2020; Gu et al. 2018; Guo et al. 2022; Lv et al. 2015].

Early studies leverage information retrieval techniques for code search [Bajracharya et al. 2014; Liu et al. 2021b; Lv et al. 2015]. These methods primarily employ unsupervised text-matching algorithms, e.g., BM25 [Robertson et al. 2009], to match queries and code snippets. However, these methods have proved insufficient for capturing deep semantics of queries and code snippets, limiting their effectiveness [Gu et al. 2018]. To better understand such semantics, prior works propose building neural models to encode queries and code snippets as embeddings and match queries and code snippets based on the similarity between their embeddings [Cheng and Kuang 2022; Gu et al. 2018; Shuai et al. 2020]. Recently, due to the impressive understanding ability of pre-trained language models (PLMs) [Feng et al. 2020; Guo et al. 2022, 2020; Wang et al. 2023c, 2021b], researchers propose fine-tuning pre-trained language models with high-quality query-code pairs for code search [Li et al. 2022; Shi et al. 2023a] and achieved state-of-the-art performance. We refer to such methods as PLM-based methods.

However, recent research shows that PLM-based methods exhibit significant performance degradation when applied to a domain, e.g., a new programming language, where they haven't been fine-tuned [Fan et al. 2024]. A straightforward remedy is to collect query-code pairs from the new domain and fine-tune the pre-trained model with these data. Specifically, query-code pairs can be either synthesized with comment-code pairs collected from code repositories or constructed manually. However, in practice, the synthesis of high-quality query-code pairs often suffers from the shortage of code comments in software projects [Briand 2003; Spinellis 2010] and the prevalence of documentation issues [Aghajani et al. 2020; Steidl et al. 2013]. Such data scarcity can be more severe for low-resource domains. On the other hand, fine-tuning PLMs requires a sufficient number of query-code pairs, and thus it is costly to construct them manually. These constraints lead to a more practical code search scenario, where we perform cross-domain code search without using any query-code pair from the target domain. We refer to this scenario as zero-shot cross-domain code search.

Due to the gap between domains, such as the distinct characteristics of various programming languages, it is challenging to develop a universal approach for zero-shot cross-domain code search. Currently, RAPID [Fan et al. 2024] is the only effective method capable of handling zero-shot cross-domain code search. It involves generating pseudo queries for code snippets from the target

domain using generative models and subsequently fine-tuning PLMs as retrieval models with the synthesized data. While this method yields satisfactory results, it presents several limitations. First, the fine-tuning process demands significant computational resources [Shi et al. 2023b]. Second, each unique domain requires a specialized model, leading to increased financial costs and management complexity. Considering the aforementioned issues, a **zero-shot, fine-tuning-free approach for cross-domain code search** would be more practical and appealing.

The challenge of zero-shot cross-domain code search lies in the domain gaps, which make the query-code mapping knowledge that code search models learned in some domains cannot be directly adopted to another domain. Thus, the key to tackling this challenging task is bridging the domain gaps. To bridge the gap, our idea is that the query-code matching process of code search, which involves both natural language understanding and programming language comprehension [Liu et al. 2021a], can be broken down into two easier tasks: query-comment matching and code-code matching. Query-comment matching refers to retrieving code comments for a given query and returning the associated code, while code-code matching denotes retrieving the target code snippet when given another code snippet example that satisfies the query. These two matching schemas operate at similar levels of abstraction, which may help reduce the reliance on domain-specific mapping knowledge, thereby potentially mitigating the impact of domain gaps.

To explore the feasibility of this idea and understand why it can work, we first conduct an empirical study in a zero-shot cross-domain scenario. We mainly focus on three key research questions (RQs):

- RQ1: How effective is query-comment matching compared to query-code matching?
- RQ2: How effective is code-code matching compared to query-code matching?
- RQ3: Can the three matching schemas complement each other?

Through the empirical study, we find that: **(1) Query-comment matching and code-code matching outperform query-code matching in certain cases. (2) There is a high degree of complementarity among the three matching schemas, i.e., query-code, query-comment, and code-code.**

Inspired by these findings, we propose **CodeBridge**, a zero-shot, fine-tuning-free approach for cross-domain code search that integrates the three matching schemas. Query-comment matching and code-code matching assume the existence of code comments and code snippet examples, respectively. However, in practical applications, code comments are often absent and there are no feasible code snippet examples for real-time queries, hindering the use of the two matching schemas. Inspired by the recent advancements of LLMs in zero-shot code summarization and code generation [Achiam et al. 2023; Li et al. 2023; Luo et al. 2023; Roziere et al. 2023], we propose leveraging LLMs to deal with these obstacles. Specifically, we first employ zero-shot prompting to guide LLMs in generating a comment for each code snippet in the codebase and producing a code for each query. Subsequently, we transform queries, code snippets, comments, and the generated code into embeddings using PLMs and assess vector similarities through three matching schemas, i.e., query-code, query-comment, and generated code-code. Lastly, we present a sampling-based fusion approach that combines the three similarity scores to rank the final results.

To evaluate CodeBridge, we further investigate the following research questions:

- RQ4: How does CodeBridge perform?
- RQ5: How effective is our fusion strategy?
- RQ6: How sensitive is CodeBridge to its components and hyper-parameters?

Experimental results show that in the zero-shot setting, our approach outperforms the state-of-the-art PLM-based code search approaches, i.e., CoCoSoDa [Shi et al. 2023a] and UniXcoder [Guo et al. 2022], by an average of 21.4% and 24.9% in MRR, respectively, across three datasets of different

domains. Moreover, our fine-tuning-free method can yield results better than or comparable to those of RAPID [Fan et al. 2024], which necessitates fine-tuning for each domain. Further analysis demonstrates the effectiveness of our fusion strategy within our approach and reveals that our approach is effective across various retrieval models, weight selections, and LLMs.

In summary, this paper makes the following contributions:

- We empirically investigate the effectiveness of query-comment matching and code-code matching in zero-shot cross-domain code search. Our analysis highlights, for the first time, the high complementarity of the query-code, query-comment, and code-code matching schemas, which can be utilized to mitigate domain gaps.
- We propose a novel fine-tuning-free approach for zero-shot cross-domain code search. Our approach is easy to implement and establishes a strong baseline for zero-shot cross-domain code search.
- We conduct extensive experiments to evaluate the effectiveness of our approach on zero-shot cross-domain code search. Our approach outperforms the state-of-the-art PLM-based code search approaches and achieves performance that is better than or comparable to existing methods for zero-shot cross-domain code search that necessitate fine-tuning.

2 Background and Related Work

2.1 Code Search

2.1.1 Problem Formulation. The code search task aims to retrieve relevant code snippets from a codebase containing a set of code snippets $\{c_1, c_2, \dots, c_n\}$ for a natural language query q . This is accomplished by ranking the code snippets based on their computed similarity scores $\{s_1, s_2, \dots, s_n\}$, where each score $s_i = f(q, c_i)$ reflects the relevance of code snippet c_i to query q . The system then returns the top- k code snippets as results.

In IR-based methods, f is commonly a text-matching function. In deep learning (DL)-based methods, a model first transforms q and c_i into vectors v_q and v_{c_i} . The function f usually computes the cosine similarity between v_q and v_{c_i} . The model performing this task is called a retriever.

2.1.2 Code Search Methods. Code search methods can be categorized into two groups: IR-based methods and DL-based methods. IR-based methods rely on text-matching algorithms to retrieve relevant code snippets but struggle to capture deep semantic meaning. In contrast, DL-based methods, which utilize neural networks to learn query-code correlations in large-scale datasets, have become increasingly popular in recent years. For example, DeepCS [Gu et al. 2018] uses recurrent neural networks to embed queries and code in a shared vector space.

Recently, pre-trained models have demonstrated superior performance compared to traditional DL-based methods [Feng et al. 2020; Guo et al. 2022, 2020; Wang et al. 2021b]. Pre-trained models first learn extensive code knowledge from large-scale code repositories and are then fine-tuned on domain-specific query-code pairs. To this end, many pre-trained code models have been proposed. [Feng et al. 2020] introduce CodeBERT, which is the first pre-trained model specifically designed for code. [Guo et al. 2022] propose UniXcoder, a unified cross-modal pre-trained model for both code-related understanding tasks and generation tasks. [Shi et al. 2023a] propose CoCoSoDa, which utilizes soft data augmentation and multimodal momentum contrastive learning to align query-code pairs, achieving state-of-the-art results on the CodeSearchNet [Husain et al. 2019] benchmark.

Our proposed approach is also built upon PLMs. However, in contrast to the work mentioned above, this work focuses on zero-shot cross-domain code search. In addition, our proposed approach can leverage existing models without modifying their internal structure or parameters and thus is complementary instead of competing with existing PLM-based methods.

2.2 Cross-Domain Code Search

Cross-domain code search refers to retrieving relevant code snippets from a target domain when the initial training data comes from different domains. IR-based methods [Lv et al. 2015; Zhang et al. 2021] utilizing unsupervised text matching algorithms can naturally handle this situation. However, they struggle to capture deep semantics. DL-based methods are more effective, but adapting DL-based methods to a new domain is challenging without sufficient labeled data. Existing methods that tackle this challenge can be divided into three categories:

Pre-training uses large-scale unlabeled data and self-supervised objectives to train neural models. The trained model can learn common knowledge from the data and be adapted to different domains. For example, [Guo et al. 2020] introduces edge prediction and node alignment as pre-training objectives to leverage data flow information. UniXcoder [Guo et al. 2022] employs a denoising objective and incorporates abstract syntax tree information in pre-training.

Meta learning uses multiple tasks to help models adapt to new domains with very few labeled data [Finn et al. 2017]. In code search, [Chai et al. 2022] apply Model-Agnostic Meta-Learning to improve model parameter initialization. [Pian et al. 2023] introduce MetaTPTrans, which learns language-agnostic information from multilingual source code.

Pseudo-labeling refers to generating labels for unlabeled data with existing generators and then training the model with the synthesized data. Pseudo-labeling has been widely used in the field of natural language processing (NLP) [Ma et al. 2021; Wang et al. 2021a]. [Fan et al. 2024] are the only ones to apply pseudo-labeling to tackle the zero-shot cross-domain code search task. They utilize pre-trained models to generate synthetic data and introduce a mixture sampling strategy to mitigate noise. RAPID exhibits outstanding performance that surpasses all baseline models.

Unlike the methods mentioned above, our approach demands no training and requires no modification of model parameters.

2.3 Large Language Models

Recently, LLMs have demonstrated impressive zero-shot capabilities in diverse code-related tasks including code generation and code summarization [Achiam et al. 2023; Guo et al. 2024; Luo et al. 2023; Nijkamp et al. 2022; Roziere et al. 2023]. *Code generation* refers to generating a code for a given natural language description. In recent years, LLMs designed for coding tasks, such as CodeGen [Nijkamp et al. 2022], Code Llama [Roziere et al. 2023] and DeepSeek-Coder [Guo et al. 2024], have achieved remarkable results in code generation. *Code summarization* refers to generating a summary in natural language for a given code. Many works [Geng et al. 2024; Sun et al. 2023] have demonstrated the effectiveness of LLMs in code summarization.

In this work, we directly utilize LLMs' zero-shot capabilities to generate code and comments for cross-domain code search. The methods mentioned above can potentially be used in our approach.

3 Empirical Study Setup

We conduct an empirical study to investigate the feasibility of leveraging query-comment matching and code-code matching to mitigate domain gaps. In this section, we will introduce the experimental setup of our empirical study.

3.1 Dataset

This work focuses on cross-domain code search scenarios. Thus, we use the Solidity dataset [Chai et al. 2022], which is commonly employed as a benchmark for the cross-domain code search task [Chai et al. 2022; Fan et al. 2024]. Solidity is a language designed for smart contracts [Wohrer and Zdun 2018] and is not included in the pre-trained data of the PLMs we use. We follow the data

split of prior work [Chai et al. 2022], where the training, validation, and test sets consist of 56,976, 4,096, and 1,000 samples, respectively. We exclusively use the test set from the Solidity dataset, which consists of 1,000 test queries and their corresponding 1,000 Solidity functions.

3.2 Research Questions

Our study is structured around the following research questions (RQs):

RQ1: How effective is query-comment matching compared to query-code matching?

Our objective is to investigate the effectiveness of query-comment matching and the relationship between query-comment matching and query-code matching. To answer RQ1, we first compare the performance of query-comment matching with query-code matching. For the query-code matching schema, we calculate the similarity between the query and the code and then order the code based on the similarity scores. For the query-comment matching schema, since the original comments of code are used as queries, we first use zero-shot prompting to guide LLMs in generating a comment for each code. Then we order the code based on similarity scores between the query and comments related to the code. We then identify the differences between the retrieval results obtained from these two search methods to determine if they complement each other. Finally, we analyze the specific scenarios in which the query-comment matching method performs better.

RQ2: How effective is code-code matching compared to query-code matching? Similar to RQ1, we first compare the performance of code-code matching with query-code matching. For the query-code matching schema, the retrieval process is identical to that described in RQ1. For the code-code matching schema, because there are no code snippets labeled as matching the test queries in either the training set or validation set, we also leverage LLMs to generate a code snippet for each query. During the retrieval process, we rank the code based on the similarity scores between the code and the generated code. We then also analyze the differences between the retrieval results from the two search methods and the specific scenarios where code-code matching performs better.

RQ3: Can the three matching schemas complement each other? Considering the complementarity between the query-code and query-comment schemas, as well as between the query-code and code-code schemas, we would like to investigate whether these three schemas can complement each other, and how their relationships can be leveraged for zero-shot code search. To address this RQ, We further analyze the retrieval results from the three matching schemas and the outcome of integrating these three schemas.

3.3 Evaluation Metrics

We follow previous studies [Chai et al. 2022; Cheng and Kuang 2022; Fan et al. 2024] and utilize two widely adopted metrics, MRR (Mean Reciprocal Rank) and top- k accuracy ($k = 1, 5, 10$), to evaluate the performance of code search approaches. MRR is the average of the reciprocal ranks of correct answers for a set of queries. Top- k accuracy is the proportion of queries for which relevant code can be found among the top k results.

3.4 Implementation Details

As UniXcoder [Guo et al. 2022] has achieved remarkable results and is widely used as a backbone model in code search [Shi et al. 2023a; Wang et al. 2023a,b], we utilize UniXcoder as the retriever to perform zero-shot code search. Following previous studies [Fan et al. 2024; Feng et al. 2020; Wang et al. 2023c], we set the maximum sequence length of the retriever's input to 256 for programming language (PL) and 128 for natural language (NL), respectively. Considering both effectiveness and efficiency, we utilize DeepSeek-Coder-1.3B-Instruct for zero-shot code and comment generation due to its impressive performance and the relatively small number of parameters. We set the maximum generation length of the LLM to 256 for PL and 128 for NL.

4 Empirical Results

4.1 RQ1: How effective is query-comment matching compared to query-code matching?

Table 1. Performance on Solidity with Different Matching Schemas

Schema	MRR	Top-1	Top-5	Top-10
Query-Code	0.544	0.452	0.651	0.701
Query-Comment	0.500	0.409	0.605	0.681
Code-Code	0.410	0.330	0.492	0.566

The results are shown in Table 1. The results demonstrate that query-comment matching is not superior to query-code matching. However, the MRR and accuracy of the query-comment approach are close to the query-code approach, suggesting promising potential for query-comment matching.

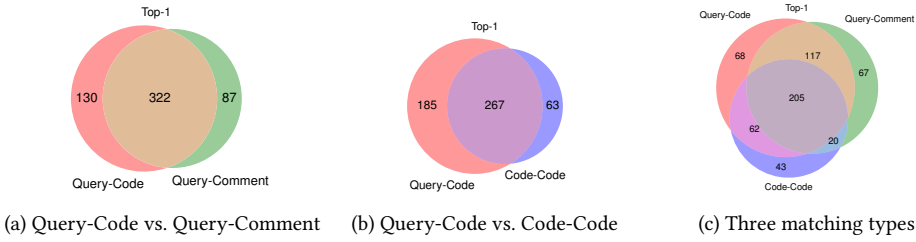


Fig. 1. The Venn diagram of the top-1 retrieved samples using Unixcoder based on the Solidity test set.

Fig. 1a presents the Venn diagram of the cases where the first code snippet retrieved by the query-code approach or the query-comment approach is correct. The result shows that the query-comment approach successfully retrieves 87 samples that cannot be correctly handled by the query-code approach, accounting for 19.2% of the top-1 retrieved samples of the query-comment approach. **This indicates that the two approaches are complementary.**

Table 2. Proportions of categories

Outperforming Pattern	#Samples	Category	Proportion
Query-Comment	87	Comments mask domain-specific implementation details	62.0%
		Comments mask unnecessary details	38.0%
Code-Code	63	Direct token matching	28.6%
		Operation sequence matching	71.4%

We analyze the 87 samples that are successfully handled by query-comment matching but not by query-code matching. We observe that when the discrepancy between the query and the code is significant, and the comment effectively summarizes the function’s purpose while aligning closely with the query’s intent, query-comment matching outperforms query-code matching. We further analyze the potential reasons for the better performance of query-comment matching on these samples. We find that in contrast to query-code matching, which requires mapping knowledge between the query and the code, the comment masks the implementation details in the code and aligns with the query at the same level of abstraction. This makes the matching process more straightforward. We classify these potential reasons into two categories, with their respective proportions outlined in Table 2 and illustrative examples provided in Fig. 2:

Case 1:	Case 2:
Query: checks if the address already invested Code: <pre>contract c39716 { function isInvestor(address who) returns(bool) { for (uint i = 0; i < investors.length; i++) if (investors[i] == who) return true; return false; } }</pre> Comment: This Solidity contract is designed to manage a list of investors. The 'isInvestor' function checks if a given address is an investor in the contract.	Query: sets time lock for given allocation address Code: <pre>contract c12429 { function setInitialAllocationTimeLock(address allocationAddress, uint32 timelockTillDate) external onlyController returns(bool) { require(allocationAddress != address(0)); require(timelockTillDate >= now); timelockedAddresses[allocationAddress] = timelockTillDate; emit InitialAllocationTimeLocked(allocationAddress, timelockTillDate); return true; } }</pre> Comment: This Solidity contract is designed to manage the allocation of funds to different addresses with a timelock. The 'setInitialAllocationTimeLock' function allows the controller to set an address and a timelock period for that address. The timelock period is in seconds and is set to expire after the current time.

Fig. 2. Cases where query-comment method outperforms query-code method in Solidity's test dataset. The code is the ground truth code for the given query. The comment is generated by LLMs.

(1) Comments mask domain-specific implementation details. In Case 1 illustrated in Fig. 2, the query-code matching model faces challenges in correlating the query “checks if the address already invested” with specific details such as the variable type *address*, the concept of investors, and the equality comparison *investors[i] == who* of type *address*. However, the comment abstracts these details and clearly states the function's intent, thereby simplifying the comparison process.

(2) Comments mask unnecessary details. In Case 2 depicted in Fig. 2, the target code matches the query “sets time lock for given allocation address”. However, the code also involves additional steps like validating the address, which are not directly relevant to the primary action described by the query. Such extra details can introduce noise when aligning queries with code. In contrast, comments concisely summarize the core functionality, reducing noise and enabling accurate matching.

By analyzing the 130 samples handled successfully by query-code matching but not query-comment matching, we observe two failure causes for query-comment matching, i.e., (1) imprecise comments and (2) excessive noise. Imprecise comments hinder the alignment between queries and comments. For example, a comment describes the code as a “CryptoKitties-like game token contract” while omitting its key function of initiating contributions. This naturally leads to a mismatch with the query “Start Contribute”. Meanwhile, excessive noise drowns out key information, weakening the match. For example, a comment accurately describes the target function “is used to add a new owner to the contract” but also includes extraneous details, such as the usage of modifiers. This additional information weakens the match between “add a new owner” and the query “Adds an owner”. These findings highlight that only relying on query-comment matching is not enough for accurate code search.

Answer to RQ1: When used individually, query-comment matching does not outperform query-code matching. However, they are complementary. Further analysis suggests that query-comment matching can perform better than query-code matching in certain scenarios. This may be because comments often omit domain-specific and/or unnecessary details in the code, reducing the need for domain-specific query-code mapping knowledge. Query-comment matching might fail when comments are imprecise or contain excessive noise.

4.2 RQ2: How effective is code-code matching compared to query-code matching?

The experimental results are shown in Table 1. The results demonstrate that code-code matching is also not superior to query-code matching. Fig. 1b presents the Venn diagram of the cases where the first code snippet retrieved by the query-code approach or the code-code approach is correct. The result shows that the code-code matching successfully retrieves 63 samples that cannot be correctly handled by the query-code approach, accounting for 13.9% of the top-1 retrieved samples of the query-code approach. **This also indicates that the two approaches are complementary.**

Case 1:	Case 2:
Query: Deposits tokens in game to some user Code: <pre> contract c10549 { function sendTo(address _user, uint64 _amount) external { require(walletBalances[msg.sender] >= _amount); walletBalances[msg.sender] -= _amount; if (userIds[_user] > 0) { balances[userIds[_user]] += _amount; } else { walletBalances[_user] += _amount; } emit Deposit(_user, _amount); } } } </pre>	Query: Allows the pendingOwner address to finalize the transfer, as long as it is called within the specified start and end time Code: <pre> contract c20691 { function claimOwnership() onlyPendingOwner public { require((block.number <= end) && (block.number >= start)); OwnershipTransferred(owner, pendingOwner); owner = pendingOwner; pendingOwner = address(0); end = 0; } } } </pre>
Generated Code: <pre> contract TokenDeposit { address public owner; mapping(address => uint256) public balances; event Deposit(address indexed user, uint256 amount); function deposit() public payable { require(msg.value > 0, "Deposit amount must be greater than " + "zero"); balances[msg.sender] += msg.value; emit Deposit(msg.sender, msg.value); } } </pre>	Generated Code: <pre> contract PendingOwner { function finalizeTransfer() public { require(now >= startTime && now <= endTime, "Transfer not within time range"); pendingOwner = address(0); } } </pre>

Fig. 3. Cases where code-code method outperforms query-code method in Solidity’s test dataset. The code represents the correct code for the query, while the generated code denotes the code generated by LLM for the same query.

Similar to RQ1, we analyze the 63 samples that are successfully handled by the code-code matching but not by the query-code matching. We find that in these cases, the generated code shares a similar implementation to the target code. The reason why code-code matching outperforms query-code matching in these samples may be attributed to the fact that the two code snippets operate within the same abstract level. Specifically, code deals with low-level operations, and directly comparing these low-level implementations can help reduce the need for domain-specific query-code mapping knowledge. We classify these potential reasons into two categories, with their respective proportions outlined in Table 2 and illustrative examples provided in Fig. 3:

(1) In case 1 illustrated in Fig. 3, both the generated code and the target code start a deposit by using “emit Deposit”, a specific syntax in Solidity. Such direct match reduces the difficulty for the model to identify the relevant answer. In contrast, the query-code matching schema fails in this case because the operation of depositing tokens is relatively rare in other programming languages like Python and understanding how to deposit tokens can be difficult without domain-specific

knowledge. This case indicates that the implementation similarities between the generated code and the target code can facilitate a more direct match and reduce the difficulties in cross-domain code search.

(2) In case 2 illustrated in Figure 3, both the generated code and the target code implement time constraints by using *require* statement and a mechanism to finalize the ownership transfer in the same order. The model can identify the comparable operation sequences directly without first learning the code's structure for a domain-specific query. However, the query-code matching schema fails in this case because the specific implementation order of operations for finalizing a transfer is difficult to determine without domain-specific knowledge. This case indicates that structure similarities between the generated code and the target code can also help reduce the need for domain-specific knowledge.

We analyze the 185 samples successfully handled by query-code matching but not by code-code matching. We identify two failure causes for code-code matching: (1) imprecise generated code and (2) differing implementations. Imprecise code often leads to mismatches. For example, the generated code that simply returns a *byte32* variable naturally fails to match the query "Extract 256-bit worth of data from the bytes stream". Differing implementations also hinder code-code matching. For example, for the query "create a new offer with setting", the generated code uses a struct *Offer* with an initialization method, while the target code uses an internal function *CreateOffer_internal*. This implementation mismatch prevents successful code-code matching.

Results in Table 1 show that code-code matching underperforms compared to query-comment matching. This may be due to the diversity of code: developers use varying identifiers, and implementations differ in abstraction levels, such as low-level instructions versus high-level library functions. These factors make code-code matching harder than simpler query-comment matching.

Answer to RQ2: When used individually, code-code matching does not surpass query-code matching in performance. However, the two schemas are also complementary. Further analysis indicates that code-code matching might outperform query-code matching in certain scenarios. The main reason is that code-code matching focuses on the same low-level operations, which reduces the need for domain-specific knowledge to map queries to code. Code-code matching might fail when the generated code is imprecise or using a different implementation.

4.3 RQ3: Can the three matching schemas complement each other?

The three matching schemas each have distinct advantages. Query-comment matching simplifies the matching process by masking implementation details. Code-code matching compares low-level implementations in the code, reducing the need for domain-specific knowledge. We further analyze the advantages of query-code matching by examining: (1) 130 samples successfully handled by query-code matching but not by query-comment matching; (2) 185 samples successfully handled by query-code matching but not by code-code matching. We also identify two specific scenarios where query-code matching outperforms the other matching schemas:

(1) **Direct function name matching:** Docstring-code pairs, where function names usually align with docstrings, are used to train code search models like CoCoSoDa. Thus, query-code matching effectively identifies relevant code snippets when the query directly matches a function name.

(2) **Function specification matching:** Query-code matching performs better when the query includes function specifications (e.g., return types, input parameters), which can be directly mapped to the code implementation for accurate matching.

Fig. 1c presents the Venn diagram of samples from the three matching schemas where the correct code is successfully ranked first. The three matching schemas exhibit high complementarity. The

query-comment matching schema retrieves 67 samples not found in the query-code and code-code retrieval samples. Meanwhile, the code-code matching schema retrieves 43 samples that are not included in the set of query-code and query-comment retrieval samples. By combining the top-1 retrieval samples from the three matching schemas, an additional 130 samples can be retrieved at the top rank, constituting 28.8% of the top-1 retrieval samples from the query-code matching schema. This underscores the significant potential of integrating these three matching schemas to enhance code search performance.

Answer to RQ3: The three matching schemas are complementary. Combining retrieval results of three matching approaches can yield a significant 28.8% enhancement in top-1 accuracy over query-to-code matching alone.

5 Approach of CodeBridge

The results of our empirical study show that query-comment matching can hide domain-specific and unnecessary details in the code, thereby enabling matching at a similar level of abstraction. Meanwhile, code-code matching facilitates a direct comparison of the concrete implementation details, thus focusing the matching on lower-level operations. Each method provides distinct advantages in specific cross-domain scenarios and can serve as a complement to query-code matching. Inspired by these findings, we propose a novel method named CodeBridge to bridge domain gaps for cross-domain code search by integrating the three matching schemas.

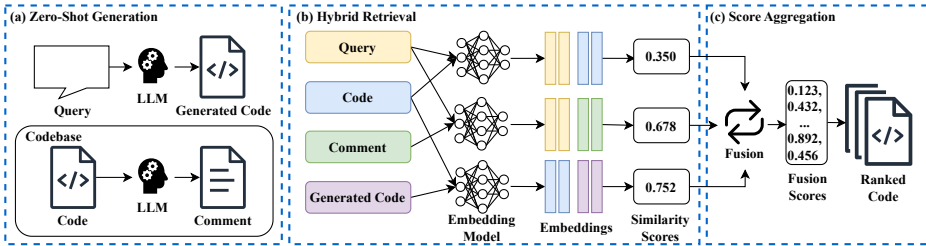


Fig. 4. Overall Framework of CodeBridge

Fig. 4 shows the overall framework of CodeBridge. CodeBridge is composed of three components: **zero-shot generation**, **hybrid retrieval**, and **score aggregation**. (1) In the zero-shot generation stage, queries are translated into code snippets, and conversely, code is translated into descriptive comments. As comments are often missing in practical applications, and the code snippets that satisfy real-time queries are also unknown during runtime, we use zero-shot prompting to guide LLMs in generating code and creating code summaries. (2) In the hybrid retrieval stage, all entities, including queries, code, comments, and generated code, are transformed into vector representations through embedding models. We then calculate the similarity scores for each matching pair. (3) In the score aggregation stage, we combine the scores and rank them to determine the final result. We propose a sampling-based fusion strategy to aggregate the scores.

5.1 Zero-Shot Generation

In this stage, we use zero-shot prompting to guide LLMs in generating comments and code. We design prompts, as presented in Table 3, for code summarization and code generation. For code summarization, we take inspiration from [Sun et al. 2023] to create our prompts. As shown in

Table 3, we include details about the programming language to help the LLM understand and tap into relevant expertise. Also, we take out the phrase “in one sentence” that is used in Sun et al.’s study [Sun et al. 2023] to make the summaries richer and more informative. For code generation, we request the LLMs to generate code in a specific language and ensure that it provides an answer.

Table 3. Prompts for Code Summarization and Code Generation

Task	Prompt
Code Summarization	Below is a {language} code that describes a task. Please give a short summary describing the purpose of the code. You must write only summary without any prefix or suffix explanations. \n {code}
Code Generation	Write a code for the following query in {language} without comments. You must return a code and must not refuse to answer. \n {query}

5.2 Hybrid Retrieval

In this stage, we calculate similarity scores in three matching schemas. After zero-shot generation, the initial query-code pair, denoted as $\langle q, c \rangle$ where q represents the query and c the source code, is expanded to $\langle q, c, m, g \rangle$. Here, m represents the comment associated with the code c , and g represents the code generated from the query q . The three matching schemas are defined as follows:

- **Query-Code Matching** ($\langle q, c \rangle$): Direct comparison of the query against the source code.
- **Query-Comment Matching** ($\langle q, m \rangle$): Comparison at an abstract level between the query and the descriptive comment.
- **Code-Code Matching** ($\langle c, g \rangle$): Evaluating the implementation consistency between the generated code snippet (g) and the original code (c).

Let $\phi(x, y)$ denote the similarity scoring function, where x and y represent the elements being compared. We first encode input into vectors using a PLM, denoted as $\mathcal{M}$. Then we calculate the similarity between vectors. We use cosine similarity by default. The formulas are as follows.

$$v_x = \mathcal{M}(x), \quad v_y = \mathcal{M}(y), \quad \phi(x, y) = \text{sim}(\mathcal{M}(x), \mathcal{M}(y)) \quad (1)$$

For each expanded $\langle q, c, m, g \rangle$ tuple, the similarity scores are computed as follows:

$$s_{qc} = \phi_{qc}(q, c), \quad s_{qm} = \phi_{qm}(q, m), \quad s_{cg} = \phi_{cg}(c, g) \quad (2)$$

where s_{type} represents the similarity score for the corresponding type of matching schema.

5.3 Score Aggregation

In this stage, we aim to aggregate the three similarity scores and rank the code snippets to obtain the final result. The most straightforward method for combining different retrieval outputs is a linear combination. However, the assignment of weights is influenced by two key aspects: the comparative matching ability of different models and the varying quality of both generated comments and synthesized code. This interplay introduces complexity in determining the ideal weights.

To obtain an appropriate weight configuration for a specific ensemble of models, we propose a sampling-based linear combination method. Consider the equation for computing the final score:

$$s_{\text{total}} = \alpha \times s_{qc} + \beta \times s_{qm} + \gamma \times s_{cg} \quad (3)$$

To determine suitable values for α , β , and γ , we first randomly select 1,000 instances from the training set of CodeSearchNet-Java [Husain et al. 2019], a widely-used dataset for code search, to serve as the validation set. These instances are all in Java, a language not represented in our

evaluation datasets. We then use grid search with a 0.05 step size to explore the full range of potential weight combinations. The configuration yielding the maximum top-10 accuracy is adopted.

6 Evaluation Settings

We conduct comprehensive experiments to evaluate the performance of CodeBridge. In this section, we first introduce the research questions. Then we describe the datasets, baselines, evaluation metrics, and implementation details.

6.1 Research Questions

We systematically evaluate the effectiveness (RQ4 and RQ5) of CodeBridge and analyze its sensitivity to its components and hyper-parameters (RQ6).

RQ4: How does CodeBridge perform? We evaluate CodeBridge across three datasets of different domains against various code search methods, including the state-of-the-art PLM-based code search approaches such as UniXcoder [Guo et al. 2022] and CoCoSoDa [Shi et al. 2023a], as well as the state-of-the-art zero-shot cross-domain code search method, RAPID [Fan et al. 2024]. As LLMs might directly serve as in-domain code search tools, we implement zero-shot LLM embedding methods for code search and evaluate their performance against CodeBridge. Additionally, we perform an ablation study to evaluate the effectiveness of combining three matching schemas.

RQ5: How effective is our fusion strategy? Existing fusion strategies can be divided into two groups: score-based (e.g., CombSUM [Fox and Shaw 1994] and CombMNZ [Fox and Shaw 1994]) and rank-based (e.g., Borda [Borda 1784], RRF [Cormack et al. 2009]). We compare our strategy with four widely-used fusion strategies [Benham and Culpepper 2017] on the Solidity test set.

RQ6: How sensitive is CodeBridge to its components and hyper-parameters? To analyze the sensitivity of CodeBridge, we conduct experiments on CodeBridge with different retrieval models, weight selections, and LLMs:

(1) *Different retrieval models*: we evaluate our framework with different retrieval models and their combinations on three datasets.

(2) *Different weight selections*: We investigate the effects of varying weights in Eq. (3) and the impact of the sampling dataset. First, we utilize the Solidity dataset to systematically explore the performance of all possible weight configurations under the constraint that $\alpha + \beta + \gamma = 1$, using a step size of 0.01. We present the findings through visual representations. Second, we obtain weights from datasets of three other widely used languages in CodeSearchNet [Husain et al. 2019]. Then, we analyze the impact of the sampling dataset.

(3) *Different LLMs*: We use various LLMs to perform zero-shot generation and test our approach on the Solidity dataset.

6.2 Datasets

Following previous work [Fan et al. 2024], we conduct experiments on three datasets: SQL [Chai et al. 2022], Solidity [Chai et al. 2022] and CoSQA [Huang et al. 2021]. Each dataset includes a set of test queries and a codebase. The SQL dataset and Solidity dataset contain code examples written in SQL and Solidity, respectively. These languages are not part of the training dataset for the PLMs we use. Instead of using code comments as queries, the CoSQA dataset includes real queries from Microsoft Bing [Bing 2024] and corresponding Python code snippets from Github [Github 2024].

The statistics of the three datasets are shown in Table 4. Recognizing that code comments are often missing or insufficient in practice, we remove all code comments in the code. Besides, we utilize real web queries as test queries within the CoSQA dataset following the initial setting of [Huang et al. 2021]. Please note that the dataset configuration of RAPID, the state-of-the-art zero-shot cross-domain code search approach, is different. They do not fully remove code comments,

and they use code comments as test queries in the CoSQA dataset. Additionally, RAPID applies a custom split to the CoSQA dataset, resulting in different dataset sizes. The CoSQA dataset consists of 20,604 queries and 6,267 code snippets, where each code snippet is accompanied by a comment. RAPID constructs three subsets: 20,000 code-only samples for training (including repeated code snippets), 602 comment-code pairs for validation, and 901 comment-code pairs for testing. RAPID generates comments for each code snippet in the training set to create the final comment-code pairs for training. To ensure a fair comparison, when comparing our approach to RAPID, we use the dataset configuration of RAPID. Importantly, the difference in dataset sizes does not affect the validity of our results, as we only compare CodeBridge and the baselines under the same setting, never cross settings.

Table 4. Statistics of the Datasets

Setting	Language	Train	Valid	Test	Codebase	w/ Comment	Use Web Query
Cross-Domain Setting	SQL	14,000	2,068	1,000	1,000	no	/
	Solidity	56,976	4,096	1,000	1,000	no	/
	CoSQA	19,604	500	500	6,267	no	yes
RAPID's Setting	SQL	14,000	2,068	1,000	1,000	no	/
	Solidity	56,976	4,096	1,000	1,000	yes	/
	CoSQA	20,000	602	901	901	no	no

6.3 Baselines

To comprehensively evaluate the performance of our approach, we broadly select traditional IR-based models, pre-trained models, zero-shot cross-domain code search methods, and zero-shot LLM embedding methods as baselines:

IR-based Models: BM25 [Robertson et al. 2009] is an enhanced text-matching algorithm based on TF-IDF, serving as a strong baseline for unsupervised code search.

Pre-trained Models: GraphCodeBERT [Guo et al. 2020] is a code pre-trained model that leverages control flow graph information during pre-training; CodeT5+ 110M [Wang et al. 2023c] incorporates tasks such as causal language modeling and text-code contrastive learning to improve embedding quality; UnixCoder [Guo et al. 2022] is trained using ASTs (Abstract Syntax Trees) and code comment data, enhancing its code understanding ability. CoCoSoDa [Shi et al. 2023a] applies multimodal momentum contrastive learning and soft data augmentation to enhance code and query representations. It achieves state-of-the-art performance on the CodeSearchNet dataset.

Zero-Shot Cross-Domain Methods: RAPID [Fan et al. 2024] uses a pre-trained generative model to generate queries for each code snippet and then fine-tunes the retrieval model using these synthesized query-code pairs. This approach achieves state-of-the-art performance in the scenario of zero-shot cross-domain code search.

Zero-Shot LLM Embedding Methods: Weighted Mean Pooling [Muennighoff 2022] computes the positional weighted average of token embeddings; Mean Pooling [Reimers 2019] computes the average of token embeddings; EOS Pooling [Reimers 2019] uses the last token's embedding; Echo Embedding [Springer et al. 2024] is the state-of-the-art zero-shot embedding method for LLMs, which repeats the input twice for richer contextual embeddings.

6.4 Evaluation Metrics and Implementation Details

We follow previous RQs and utilize MRR and top- k accuracy ($k = 1, 5, 10$) as the evaluation metrics.

We employ the most advanced models tailored for each type of matching schema in our approach. Specifically, we employ CoCoSoDa for query-code, BGE [Xiao et al. 2024] for query-comment, and UniXcoder for code-code matching. BGE is a superior natural language vector model trained on 300

million pairs of natural language texts, demonstrating powerful natural language representation capabilities. Specifically, we utilize the bge-large-en-v1.5 (326M). The weights in Eq. (3) of this particular model configuration are $\alpha = 0.65, \beta = 0.25, \gamma = 0.10$. We set the maximum sequence length of the retriever’s input to 256 for PL and 128 for NL, respectively.

We use DeepSeek-Coder-1.3B-Instruct for zero-shot generation and set the maximum generation length to 256 for PL and 128 for NL. For RAPID, we directly copy its results from its original paper [Fan et al. 2024]. For PLM-based methods, we use their official implementation. As IR-based methods are sensitive to pre-processing methods, we follow [Zhang et al. 2021] for BM25 preprocessing steps. We apply LLM embedding methods directly to DeepSeek-Coder-1.3B-Instruct, generating query and code embeddings for code search.

7 Evaluation Results

7.1 RQ4: How does CodeBridge perform?

The experimental results are shown in Table 5 and Table 6, which demonstrate CodeBridge’s effectiveness in all the evaluated languages and settings.

Table 5. Comparison with Baselines on Cross-domain Setting

Model	SQL				Solidity				CoSQA			
	MRR	Top-1	Top-5	Top-10	MRR	Top-1	Top-5	Top-10	MRR	Top-1	Top-5	Top-10
BM25	0.469	0.341	0.614	0.725	0.475	0.391	0.565	0.624	0.183	0.110	0.254	0.312
GraphCodeBERT	0.177	0.120	0.215	0.286	0.196	0.135	0.249	0.310	0.089	0.040	0.130	0.190
CodeT5+ 110M	0.512	0.385	0.662	0.758	0.424	0.339	0.508	0.579	0.407	0.260	0.586	0.678
UniXcoder	0.744	0.632	0.887	0.938	0.544	0.452	0.651	0.701	0.376	0.256	0.512	0.620
CoCoSoDa	0.555	0.441	0.690	0.769	0.625	0.541	0.728	0.788	0.482	0.346	0.630	0.724
Weighted Mean Pooling	0.115	0.077	0.138	0.180	0.051	0.027	0.060	0.082	0.007	0.0	0.008	0.016
Mean Pooling	0.092	0.054	0.113	0.156	0.048	0.019	0.064	0.088	0.006	0.002	0.004	0.008
EOS Pooling	0.006	0.002	0.002	0.005	0.007	0.001	0.003	0.008	0.004	0.002	0.002	0.008
Echo Embedding	0.064	0.044	0.077	0.092	0.059	0.037	0.070	0.093	0.046	0.024	0.058	0.088
Query-Code	0.555	0.441	0.690	0.769	0.625	0.541	0.728	0.788	0.482	0.346	0.630	0.724
Query-Comment	0.744	0.644	0.873	0.927	0.554	0.460	0.661	0.735	0.454	0.322	0.604	0.728
Code-Code	0.697	0.602	0.817	0.875	0.410	0.330	0.492	0.566	0.303	0.200	0.406	0.522
CodeBridge	0.811	0.723	0.929	0.958	0.658	0.571	0.762	0.833	0.544	0.424	0.682	0.782

According to Table 5, CodeBridge surpasses CoCoSoDa by an average of 21.4% in MRR and 30.6% in top-1 accuracy. Additionally, our approach outperforms UniXcoder with an average improvement of 24.9% in MRR and 35.4% in top-1 accuracy. While CoCoSoDa performs well across most languages, it doesn’t work as effectively on SQL dataset, which indicates its limitation on generalization to different domains. Meanwhile, all zero-shot LLM embedding methods perform poorly. This may be because the unidirectional causal mask in LLMs hinders bidirectional understanding, and unlike natural language texts, source code is highly structured, which may make the text embedding method not well-suited for code search. In contrast, CodeBridge exhibits consistent and superior performance across the three datasets, highlighting its great cross-domain code search capabilities.

We compare CodeBridge with RAPID under its setting, as shown in Table 6. The performance of CodeBridge is comparable to that of RAPID on the Solidity and CoSQA datasets. CodeBridge even outperforms CoCoSoDa with RAPID by 3.3% in MRR and 3.3% in top-1 accuracy on the SQL dataset. It is noteworthy that while RAPID requires collecting code snippets in the target domain and fine-tuning with synthesized data, while **our approach demands no training, and requires no intervention on the model, and thus is easy to implement and use**. Thus, we believe our approach provides unique benefits for zero-shot cross-domain code search.

Table 6. Comparison with baselines on RAPID’s setting. The best result is bolded, and the second best is underlined.

Model	SQL				Solidity				CoSQA			
	MRR	Top-1	Top-5	Top-10	MRR	Top-1	Top-5	Top-10	MRR	Top-1	Top-5	Top-10
GraphCodeBERT	0.175	0.126	0.211	0.257	0.292	0.247	0.336	0.369	0.500	0.377	0.649	0.740
+RAPID	0.723	0.628	0.837	0.896	0.767	0.710	0.830	0.868	0.899	0.857	0.956	0.977
UniXcoder	0.744	0.632	0.887	<u>0.938</u>	0.690	0.623	0.762	0.809	0.807	0.737	0.895	0.937
+RAPID	<u>0.790</u>	<u>0.713</u>	0.880	0.923	0.779	0.723	0.848	0.876	<u>0.897</u>	0.846	0.966	0.980
CoCoSoDa	0.559	0.448	0.691	0.766	0.753	0.692	0.822	0.866	0.876	0.826	0.945	0.969
+RAPID	0.785	0.700	<u>0.890</u>	<u>0.938</u>	0.789	0.739	0.853	<u>0.878</u>	0.894	<u>0.848</u>	0.966	0.980
CodeBridge	0.811	0.723	0.929	0.958	<u>0.788</u>	<u>0.735</u>	<u>0.850</u>	0.891	0.895	<u>0.848</u>	<u>0.960</u>	<u>0.979</u>

We also conduct an ablation study to compare the performance of the three matching schemas. As shown in Table 5, the results indicate that CodeBridge, which integrates all three schemas, outperforms each matching schema by substantial margins. Specifically, the average improvements of CodeBridge over query-code, query-comment, and code-code matching in MRR are 21.4%, 15.9%, and 52.1%, respectively, across the three datasets. These results suggest the effectiveness of fusing the three matching schemas, consistent with our empirical findings.

Answer to RQ4: CodeBridge significantly outperforms existing zero-shot code search baselines by 21.4% to 24.9% on average in terms of MRR. Additionally, our fine-tuning-free approach yields results that are comparable to or even better than the state-of-the-art zero-shot cross-domain code search method RAPID, which requires fine-tuning. The ablation study confirms the effectiveness of fusing the three matching schemas.

7.2 RQ5: How effective is our fusion strategy?

Table 7. Results of different strategies on Solidity dataset. Since CombSUM and CombMNZ calculate the frequency of occurrences in different result lists, they will reduce to an equally weighted linear combination when the recall size equals the total size of the dataset. Thus, set a recall of 10 for CombSUM and CombMNZ.

Fusion Strategy	CombSUM	CombMNZ	RRF	Borda	Our Strategy
Top-1	0.523	0.523	0.494	0.480	0.571
Top-5	0.748	0.748	0.705	0.655	0.762
Top-10	0.803	0.803	0.799	0.727	0.833

Table 7 presents the performance of our fusion strategy. While score-based methods outperform rank-based methods, our strategy outperforms the four widely-used fusion strategies by substantial margins, demonstrating the effectiveness of our fusion strategy. Specifically, our strategy outperforms the best-performing baselines, i.e., CombSUM and CombMNZ, by 9.2% in terms of top-1 accuracy. These results can be attributed to our strategy’s comprehensive consideration of both the model’s semantic understanding ability and the quality of generated content.

Answer to RQ5: The proposed fusion strategy outperforms four widely-used fusion strategies by substantial margins.

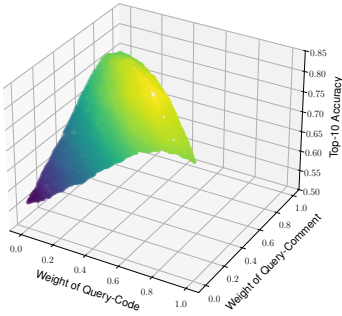
7.3 RQ6: How sensitive is CodeBridge to its components and hyper-parameters?

7.3.1 Different Retrieval Models. Based on the results shown in Table 8, we find that: (1) **Integrating tailored models for each matching schema contributes significantly to the effectiveness of**

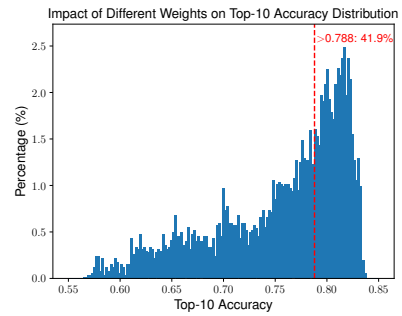
Table 8. Performance comparison with different retrieval models. "Uni" denotes UniXcoder, and "Co" denotes CoCoSoDa. Each component in the combination of models is sequentially used for query-code matching, query-comment matching, and code-code matching. CodeBridge integrates UniXcoder, BGE, and CoCoSoDa. The best result is bolded, and the second best is underlined.

Model	SQL				Solidity				CoSQA			
	MRR	Top-1	Top-5	Top-10	MRR	Top-1	Top-5	Top-10	MRR	Top-1	Top-5	Top-10
UniXcoder	0.744	0.632	0.887	0.938	0.544	0.452	0.651	0.701	0.376	0.256	0.512	0.620
Uni-Uni-Uni	0.795	0.697	0.914	0.957	0.581	0.491	0.687	0.737	0.426	0.296	0.572	0.696
Uni-BGE-Uni	0.824	0.737	0.940	0.973	0.640	0.559	0.736	0.800	0.513	0.382	0.654	0.776
CoCoSoDa	0.555	0.441	0.690	0.769	0.625	0.541	0.728	0.788	0.482	0.346	0.630	0.724
Co-Co-Co	0.592	0.478	0.732	0.804	0.635	0.547	0.747	0.794	0.497	0.364	0.644	0.744
Co-BGE-Co	0.779	0.687	0.899	0.934	<u>0.657</u>	0.572	0.765	<u>0.820</u>	<u>0.525</u>	<u>0.390</u>	<u>0.676</u>	<u>0.772</u>
CodeBridge	<u>0.811</u>	<u>0.723</u>	<u>0.929</u>	<u>0.958</u>	0.658	<u>0.571</u>	<u>0.762</u>	0.833	0.544	0.424	0.682	0.782

our approach. Specifically, models incorporating BGE (i.e., Uni-BGE-Uni and Co-BGE-Co) surpass those without BGE (i.e., Uni-Uni-Uni and Co-Co-Co). For example, Uni-BGE-Uni outperforms Uni-Uni-Uni by an average of 11.4% in MRR and Co-BGE-Co outperforms Co-Co-Co by an average of 13.6% in MRR. This can be attributed to the superior natural language understanding ability of NL embedding models, which provides more accurate query-comment matching than code pre-trained models. CodeBridge also surpasses Uni-BGE-Uni on two datasets, highlighting the advantage of using an advanced code search model like CoCoSoDa for query-code matching. In addition, CodeBridge's performance is better than or comparable to Co-BGE-Co, indicating that using UniXcoder, which performs outstandingly in code-to-code search tasks, contributes to the effectiveness of our method. (2) **Our approach exhibits consistent performance across different retrieval models.** All variants outperform their corresponding base models, i.e., UniXcoder and CoCoSoDa, by substantial s. For example, Uni-Uni-Uni outperforms UniXcoder with an average MRR increase of 9.0%, and Co-Co-Co achieves an average MRR enhancement of 3.8% over CoCoSoDa.



(a) Visualizing the Effects of Weight Adjustments on Top-10 Accuracy



(b) Weight Distribution and Corresponding Top-10 Accuracy

Fig. 5. Impact of Weight variations on Top-10 Accuracy: Visualization Results

7.3.2 Impact of Weight Selection. Fig. 5a illustrates the variation in top-10 accuracy resulting from different weight adjustments of CodeBridge on the Solidity dataset. It can be observed that the variation of top-10 accuracy generally exhibits a semi-peak profile and the peak is not sharp. This suggests that suitable weights fall within a broad range rather than being critically sensitive to minute changes. Fig. 5b shows the distribution of the weights that result in various ranges of top-10

accuracy. Please note that CoCoSoDa achieves a zero-shot performance of 0.788 on top-10 accuracy. The result shows that 41.9% of weight choices can produce outcomes that exceed CoCoSoDa. This observation is consistent with the data presented in Fig. 5a. These results suggest that our approach demonstrates robustness in weight selection.

Table 9. Performance on the Solidity Dataset with Different Sampling Datasets

Sampling Dataset	MRR	Top-1	Top-5	Top-10
Java/Go/Python	0.658	0.571	0.762	0.833
JavaScript	0.663	0.579	0.766	0.828

Table 9 illustrates the performance of our approach with different sample datasets on the Solidity dataset. Our approach achieves consistent performance with different sampling datasets. The optimal weights for the Java, Go, and Python datasets are identical, and our approach achieves even better results with the optimal weight on the JavaScript dataset. This indicates that our approach is not sensitive to the sampling datasets.

Table 10. Performance on the Solidity Dataset with Different LLMs

LLM	MRR	Top-1	Top-5	Top-10
DeepSeek-Coder-1.3B-Instruct	0.658	0.571	0.762	0.833
DeepSeek-Coder-6.7B-Instruct	0.664	0.580	0.771	0.824
CodeLlama-7B-Instruct	0.668	0.584	0.770	0.827
CodeQwen1.5-7B-Chat	0.663	0.576	0.767	0.825

7.3.3 Different LLMs. Table 10 shows that our approach performs slightly differently with different LLMs, which is attributed to that the zero-shot generation capabilities of different LLMs differ. It is worth mentioning that our approach outperforms the state-of-the-art baseline CoCoSoDa with different LLMs. These results indicate that our approach is not sensitive to the used LLMs.

Answer to RQ6: Our experiments demonstrate that CodeBridge can perform well across different retrieval models, weight selections, and LLMs.

8 Discussion

8.1 Data Leakage

LLMs have been trained on open-source data and may have seen the test samples in our experiments. Utilizing LLMs to generate comments and code results in a possibility of data leakage. To mitigate data leakage threats, we additionally collect a dataset containing 1000 Rust samples from GitHub repositories created after February 2023 (the cutoff date for the pre-training data of DeepSeek-Coder). We choose Rust, a language excluded from the embedding model's pre-training data, to ensure a cross-domain setting. Specifically, we (1) collect 82 non-fork repositories with the largest number of stars; (2) apply the data filtering principles from GraphCodeBERT [Guo et al. 2020], e.g., removing samples with too short or too long queries; and (3) randomly select 1000 samples from the collected 6331 functions.

The results show that CodeBridge achieves an MRR of 0.664, outperforming UniXcoder and CoCoSoDa by 15.7% and 8.1%, respectively. It also outperforms the query-code, query-comment, and code-code matching schemas by 8.1%, 23.2%, and 87.6%, respectively. The results are consistent with those reported in Section 7.1 across the three datasets. Therefore, we believe that the threat of data leakage is limited. Full results can be found in [CodeBridge 2024].

8.2 Computational Efficiency

To better understand the benefits of CodeBridge and the trade-offs between training and inference overhead, we analyze the computational overhead of CodeBridge compared to the fine-tuning-based approach RAPID (based on UniXcoder). Both RAPID and CodeBridge require offline comment generation, offline codebase processing, online embedding model inference, and online embedding retrieval. However, CodeBridge leverages LLMs to generate code in real time for each query, introducing additional LLM inference overhead, while RAPID fine-tunes UniXcoder with code and generated comments in the target domain, resulting in additional training overhead. As offline comment generation and offline codebase processing are required by both models and are one-time efforts, we focus on their key differences, i.e., training overhead and LLM inference overhead. We also report their online computational overheads to better understand their online efficiency. We use the CoSQA dataset because it has the largest codebase among the three datasets, making it more representative of real-world applications. During training and inference, the maximum lengths are set to 128 for queries and 256 for code. We conduct training using UniXcoder’s official training script and utilize vLLM [Kwon et al. 2023] for LLM inference. All experiments are performed on one single A800 80GB GPU. We evaluate two batch sizes, i.e., 64 (the default batch size for UniXcoder training) and 256 (the default maximum batch size for vLLM). To efficiently handle embedding model inference and retrieval, we employ multiprocessing in these two stages.

Table 11. Training and inference overhead of different models

Model	Batch Size	Training (min)	LLM Inference (ms/sample)	Embedding Model Inference (ms/sample)	Retrieval (ms/sample)
RAPID (UniXcoder)	64	129.0	—	2.7	2.7
CodeBridge		—	36.0	3.2	2.8
RAPID (UniXcoder)	256	123.0	—	2.6	2.7
CodeBridge		—	24.0	3.0	2.7

Table 11 shows the computational overhead of RAPID and CodeBridge at each stage. The differences in embedding model inference overhead and retrieval overhead between the two models are negligible (less than 1 millisecond per sample). The primary difference lies in the training overhead of RAPID and the inference overhead of LLM. With a batch size of 64, the training of RAPID on one target domain takes 129.0 minutes, and the LLM inference latency of CodeBridge is 36.0 milliseconds per sample, which means the training overhead of RAPID can be used to infer 215,000 samples with the LLM. Increasing the batch size to 256 further raises this capacity to 307,500 samples, but does not significantly speed up embedding model inference and retrieval, as the GPU utilization is already near its maximum. Prior work suggests that a web page response time of less than 2 seconds is considered acceptable for most users [Lohr 2012]. The average inference latency of CodeBridge is only about 42 milliseconds per sample. Thus we believe it is acceptable for daily use. With the continuous and rapid advancements in LLM inference acceleration technologies [Kwon et al. 2023; Zhao et al. 2024], the inference latency of LLM can be further reduced in the near future.

While fine-tuning-based methods like RAPID might be cost-effective for a highly popular domain, real-world queries often cover multiple domains (e.g., programming languages) [Guo et al. 2024; Sheng et al. 2021]. In such multi-domain scenarios, our one-fit-all framework does not need to maintain and update multiple models for different domains, which is a well-known challenge in machine learning operations and introduces additional costs [Schelter et al. 2015]. CodeBridge also benefits less frequently accessed domains. Due to the long-tail distribution [Index 2025], these domains account for the majority and make up a significant share of use cases. For these domains,

using CodeBridge is more cost-effective as it eliminates the need for fine-tuning and managing one model for each domain. Thus, we believe CodeBridge provides unique benefits.

8.3 Code-Comment Pattern

CodeBridge utilizes both generated code and comments. The code-comment matching, which retrieves comments for the code generated from a query and then returns the associated code, may also be used to improve code search. To investigate this idea, we evaluate code-comment matching using CoCoSoDa, the state-of-the-art embedding model for code search. Experimental results show that code-comment matching performs worst among the four matching schemas on the SQL and CoSQA datasets and second worst on the Solidity dataset, with a relative margin of only 1.5% over the worst schema. When combining the four matching schemas, grid search reveals that the optimal weights for code-comment matching are consistently zero across all three datasets. Using the sampling method in Section 5.3 to combine the four schemas, the new CodeBridge achieves comparable results to the original CodeBridge on Solidity and CoSQA but suffers a 2.1% MRR drop on SQL, likely due to the noise in the generated code and comments. Therefore, this matching schema does not bring significant benefits considering its performance and additional overhead. The full results are available at [CodeBridge 2024].

9 Threats to Validity

The generalizability of our experimental results is a threat to the validity of this study. Due to practical constraints, we are unable to assess the cross-domain performance of our approach across all languages. However, we test three languages, including two less common ones (SQL and Solidity), which are not part of the languages in the training data for the PLMs we use. Experimental results show that our approach exhibits consistent performance across the three datasets, showcasing its language-agnostic nature and demonstrating its applicability across various domains.

10 Conclusion

In this paper, we conduct an empirical study to reveal the effectiveness of query-comment matching and code-code matching in zero-shot cross-domain code search for the first time. The empirical results reveal the high complementarity among three matching schemas, i.e., query-code, query-comment, and code-code matchings. Based on the empirical findings, we propose CodeBridge, a zero-shot cross-domain code search approach without fine-tuning. It first leverages LLMs to generate comments and code to expand the query-code pairs and then integrates the three matching schemas. We also propose a sampling-based fusion approach to combine the three similarity scores to rank the code. Experimental results show that CodeBridge outperforms the state-of-the-art PLM-based code search approaches and yields results that are better than or comparable to those of the zero-shot cross-domain code search technique RAPID, which requires fine-tuning. In the future, we plan to enhance the generation quality for retrieval and ways to improve efficiency.

11 Data Availability

Our replication package, including datasets and source code, is available at [CodeBridge 2024].

Acknowledgments

This research/project is supported by Zhejiang Provincial Natural Science Foundation of China (No. LZ25F020003), the National Natural Science Foundation of China (No. 62202420, No. 62202074), and the National Research Foundation, under its Investigatorship Grant (NRF-NRFI08-2022-0002). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- Emad Aghajani, Csaba Nagy, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, Michele Lanza, and David C Shepherd. 2020. Software documentation: the practitioners' perspective. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 590–601. doi:10.1145/3377811.3380405
- Sushil Bajracharya, Joel Ossher, and Cristina Lopes. 2014. Sourcerer: An infrastructure for large-scale collection and analysis of open-source code. *Science of Computer Programming* 79 (2014), 241–259. doi:10.1016/j.scico.2012.04.008
- Rodger Benham and J Shane Culpepper. 2017. Risk-reward trade-offs in rank fusion. In *Proceedings of the 22nd Australasian Document Computing Symposium*. 1–8. doi:10.1145/3166072.3166084
- Bing. 2024. Bing Search Engine. <https://bing.com/>. [Accessed 26-08-2024].
- JC Borda. 1784. Mémoire sur les élections au scrutin, Histoire de l'Académie royale des sciences pour 1781. *Paris (English Transl. by Grazia, A. 1953. Isis 44)* (1784).
- Lionel C Briand. 2003. Software documentation: how much is enough?. In *Seventh European Conference on Software Maintenance and Reengineering, 2003. Proceedings. IEEE*, 13–15. doi:10.1109/csmr.2003.1192406
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- Yitian Chai, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Cross-domain deep code search with meta learning. In *Proceedings of the 44th International Conference on Software Engineering*. 487–498. doi:10.1145/3510003.3510125
- Yi Cheng and Li Kuang. 2022. CSRS: code search with relevance matching and semantic matching. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. 533–542. doi:10.1145/3524610.3527889
- CodeBridge. 2024. Replication Package. <https://github.com/ZJU-CTAG/CodeBridge>.
- Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 758–759. doi:10.1145/1571941.1572114
- Guodong Fan, Shizhan Chen, Cuiyun Gao, Jianmao Xiao, Tao Zhang, and Zhiyong Feng. 2024. Rapid: Zero-shot Domain Adaptation for Code Search with Pre-trained Models. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–35. doi:10.1145/3641542
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*. PMLR, 1126–1135.
- Edward Fox and Joseph Shaw. 1994. Combination of multiple searches. *NIST special publication SP* (1994), 243–243. doi:10.6028/nist.sp.500-225.vpi
- Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3608134
- Github. 2024. Github website. <https://github.com/>. [Accessed 26-08-2024].
- Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*. 933–944. doi:10.1145/3180155.3180167
- Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7212–7225. doi:10.18653/v1/2022.acl-long.499
- Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- Junjie Huang, Duyu Tang, Linjun Shou, Ming Gong, Ke Xu, Daxin Jiang, Ming Zhou, and Nan Duan. 2021. CoSQA: 20,000+ Web Queries for Code Search and Question Answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 5690–5700. doi:10.18653/v1/2021.acl-long.442
- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

- TIOBE Index. 2025. TIOBE Index for January 2025. <https://www.tiobe.com/tiobe-index/>. [Accessed 08-02-2025].
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626. doi:10.1145/3600006.3613165
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. 2022. Coderetriever: A large scale contrastive pre-training method for code search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 2898–2910. doi:10.18653/v1/2022.emnlp-main.187
- Chao Liu, Xin Xia, David Lo, Cuiyun Gao, Xiaohu Yang, and John Grundy. 2021a. Opportunities and challenges in code search tools. *ACM Computing Surveys (CSUR)* 54, 9 (2021), 1–40. doi:10.1145/3480027
- Chao Liu, Xin Xia, David Lo, Zhiwei Liu, Ahmed E Hassan, and Shanping Li. 2021b. Codematcher: Searching code based on sequential semantics of important query words. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–37.
- Steve Lohr. 2012. For impatient web users, an eye blink is just too long to wait. *The New York Times* (2012), A1–L.
- Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568* (2023).
- Fei Lv, Hongyu Zhang, Jian-guang Lou, Shaowei Wang, Dongmei Zhang, and Jianjun Zhao. 2015. Codehow: Effective code search based on api understanding and extended boolean model (e). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 260–270. doi:10.1109/ase.2015.42
- Ji Ma, Ivan Korotkov, Yinfei Yang, Keith Hall, and Ryan McDonald. 2021. Zero-shot Neural Passage Retrieval via Domain-targeted Synthetic Question Generation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 1075–1088. doi:10.18653/v1/2021.eacl-main.92
- Niklas Muennighoff. 2022. Sgpt: Gpt sentence embeddings for semantic search. *arXiv preprint arXiv:2202.08904* (2022).
- Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
- Arkil Patel, Siva Reddy, Dzmitry Bahdanau, and Pradeep Dasgi. 2024. Evaluating In-Context Learning of Libraries for Code Generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2908–2926.
- Weiguo Pian, Hanyu Peng, Xunzhu Tang, Tiezhu Sun, Haoye Tian, Andrew Habib, Jacques Klein, and Tegawendé F Bissyandé. 2023. MetaTPTrans: A meta learning approach for multilingual code representation learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5239–5247. doi:10.1609/aaai.v37i4.25654
- N Reimers. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv preprint arXiv:1908.10084* (2019). doi:10.18653/v1/d19-1410
- Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389. doi:10.1561/15000000019
- Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- Saksham Sachdev, Hongyu Li, Sifei Luan, Seohyun Kim, Koushik Sen, and Satish Chandra. 2018. Retrieval on source code: a neural code search. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 31–41. doi:10.1145/3211346.3211353
- Sebastian Schelter, Felix Biessmann, Tim Januschowski, David Salinas, Stephan Seufert, and Gyuri Szarvas. 2015. On challenges in machine learning model management. (2015).
- Xiang-Rong Sheng, Liqin Zhao, Guorui Zhou, Xinyao Ding, Binding Dai, Qiang Luo, Siran Yang, Jingshan Lv, Chi Zhang, Hongbo Deng, et al. 2021. One model to serve all: Star topology adaptive recommender for multi-domain ctr prediction. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 4104–4113. doi:10.1145/3459637.3481941
- Ensheng Shi, Yanlin Wang, Wenchao Gu, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023a. Cocosoda: Effective contrastive learning for code search. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2198–2210. doi:10.1109/icse48619.2023.00185
- Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023b. Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond. In *Proceedings of the 32nd ACM SIGSOFT*

- International Symposium on Software Testing and Analysis*. 39–51. doi:10.1145/3597926.3598036
- Jianhang Shuai, Ling Xu, Chao Liu, Meng Yan, Xin Xia, and Yan Lei. 2020. Improving code search with co-attentive representation learning. In *Proceedings of the 28th International Conference on Program Comprehension*. 196–207.
- Diomidis Spinellis. 2010. Code documentation. *IEEE software* 27, 4 (2010), 18–19. doi:10.1109/ms.2010.95
- Jacob Mitchell Springer, Suhas Kotha, Daniel Fried, Graham Neubig, and Aditi Raghunathan. 2024. Repetition improves language model embeddings. *arXiv preprint arXiv:2402.15449* (2024).
- Daniela Steidl, Benjamin Hummel, and Elmar Juegens. 2013. Quality analysis of source code comments. In *2013 21st international conference on program comprehension (icpc)*. Ieee, 83–92. doi:10.1109/icpc.2013.6613836
- Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Quanjun Zhang, et al. 2023. Automatic code summarization via chatgpt: How far are we? *arXiv preprint arXiv:2305.12865* (2023).
- Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. 2023a. One adapter for all programming languages? adapter tuning for code search and summarization. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 5–16. doi:10.1109/icse48619.2023.00013
- Kexin Wang, Nandan Thakur, Nils Reimers, and Iryna Gurevych. 2021a. GPL: Generative pseudo labeling for unsupervised domain adaptation of dense retrieval. *arXiv preprint arXiv:2112.07577* (2021). doi:10.18653/v1/2022.naacl-main.168
- Yanlin Wang, Lianghong Guo, Ensheng Shi, Wenqing Chen, Jiachi Chen, Wanjun Zhong, Menghan Wang, Hui Li, Hongyu Zhang, Ziyu Lyu, et al. 2023b. You Augment Me: Exploring ChatGPT-based Data Augmentation for Semantic Code Search. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 14–25. doi:10.1109/icsme58846.2023.00014
- Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023c. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 1069–1088. doi:10.18653/v1/2023.emnlp-main.68
- Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021b. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 8696–8708. doi:10.18653/v1/2021.emnlp-main.685
- Maximilian Wohrer and Uwe Zdun. 2018. Smart contracts: security patterns in the ethereum ecosystem and solidity. In *2018 International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*. IEEE, 2–8.
- Xin Xia, Lingfeng Bao, David Lo, Pavneet Singh Kochhar, Ahmed E Hassan, and Zhenchang Xing. 2017. What do developers search for on the web? *Empirical Software Engineering* 22 (2017), 3149–3185. doi:10.1007/s10664-017-9514-4
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-Pack: Packed Resources For General Chinese Embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 641–649.
- Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484. doi:10.18653/v1/2023.emnlp-main.151
- Xinyu Zhang, Ji Xin, Andrew Yates, and Jimmy Lin. 2021. Bag-of-Words Baselines for Semantic Code Search. In *Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog 2021)*. 88–94. doi:10.18653/v1/2021.nlp4prog-1.10
- Yao Zhao, Zhitian Xie, Chen Liang, Chenyi Zhuang, and Jinjie Gu. 2024. Lookahead: An inference acceleration framework for large language model with lossless generation accuracy. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6344–6355. doi:10.1145/3637528.3671614

Received 2025-02-25; accepted 2025-04-01